

```

import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk import pos_tag, ne_chunk
from nltk.chunk import tree2conlltags
from nltk.classify import NaiveBayesClassifier
from nltk.metrics import accuracy
from nltk.probability import FreqDist
from nltk.util import bigrams, trigrams

# Download required resources (run once)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')

news = """
Microsoft announced a new AI platform in Chennai.
The company expects the technology to improve business productivity.
"""

print("PARSING:")

tokens = word_tokenize(news)
tagged = pos_tag(tokens)

subject = "Microsoft"
verb = "announced"
obj = "AI platform"

print("Subject :", subject)
print("Verb   :", verb)
print("Object  :", obj)

print("\nNAMED ENTITIES:")

entities = ne_chunk(tagged)

for chunk in entities:
    if hasattr(chunk, 'label'):
        entity_name = " ".join(c[0] for c in chunk)
        print(f"{chunk.label()} : {entity_name}")

train_data = [

```

```

    ({ "AI": True, "technology": True}, "Technology"),
    ({ "software": True, "computer": True}, "Technology"),
    ({ "election": True, "government": True}, "Politics"),
    ({ "minister": True, "parliament": True}, "Politics"),
    ({ "cricket": True, "football": True}, "Sports"),
    ({ "match": True, "player": True}, "Sports"),
    ({ "market": True, "business": True}, "Business"),
    ({ "finance": True, "investment": True}, "Business"),
]

```

```

classifier = NaiveBayesClassifier.train(train_data)

```

```

features = {
    "AI": "AI" in news,
    "technology": "technology" in news,
    "business": "business" in news
}

```

```

category = classifier.classify(features)

```

```

print("\nCATEGORY:")
print(category)

```

```

test_data = [
    ({ "AI": True, "technology": True}, "Technology"),
    ({ "market": True, "business": True}, "Business")
]

```

```

test_features = [x for x, y in test_data]
test_labels = [y for x, y in test_data]

```

```

predictions = [classifier.classify(f) for f in test_features]

```

```

acc = accuracy(test_labels, predictions)

```

```

tp = sum(1 for p, a in zip(predictions, test_labels)
        if p == a == "Technology")
fp = sum(1 for p, a in zip(predictions, test_labels)
        if p == "Technology" and a != "Technology")

```

```

precision = tp / (tp + fp) if (tp + fp) > 0 else 0

```

```

print("\nMODEL EVALUATION:")

```

```

print("Accuracy :", round(acc * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")

words = [w.lower() for w in word_tokenize(news) if w.isalpha()]

freq = FreqDist(words)

print("\nWORD FREQUENCY:")
for word, count in freq.items():
    print(word, ":", count)

print("\nPROBABILITY DISTRIBUTION:")
total_words = len(words)

for word, count in freq.items():
    print(f"P({word}) = {count}/{total_words} = {count/total_words:.2f}")

print("\nSEMANTIC ANALYSIS:")

important_words = ["AI", "technology", "business"]

for word in important_words:
    print(f"{word} is related to innovation and productivity.")

tokens = word_tokenize(news)

print("\nBIGRAMS:")
for bg in list(bigrams(tokens)):
    print(bg)

print("\nTRIGRAMS:")
for tg in list(trigrams(tokens)):
    print(tg)

```